SUDOKU VARIATION WITH DIAGONAL CONSTRAINTS

Backtracking, Constraint Propagation and Pruning – Technical Report

1. Introduction

Diagonal Sudoku is a variation of the standard 9×9 Sudoku puzzle. It retains all normal Sudoku rules and adds two extra constraints: every number from 1 to 9 must occur exactly once on the main diagonal and exactly once on the anti-diagonal. These additional constraints reduce the set of valid assignments but also introduce interactions between cells that are not in the same ordinary row, column, or 3×3 box.

2. Additional Diagonal Constraints

For a cell (r,c), the main diagonal condition applies when r = c. The anti-diagonal condition applies when r + c = 8 using zero-based indexing. A diagonal cell must therefore satisfy the standard row, column and box restrictions plus the relevant diagonal restriction. The center cell (row 5, column 5 in one-based indexing) belongs to both diagonals, so its value must be unique on both diagonals simultaneously.

3. Interaction with Standard Sudoku Rules

The diagonal constraints do not replace standard Sudoku rules; they add constraints to them. For example, a candidate may be allowed by its row, column and box but rejected because the same number already appears on its diagonal. This extra pruning can make some puzzles easier to search because fewer candidates remain, although the general problem remains computationally difficult.

4. Backtracking Strategy

The solver maintains sets of values already used in each row, column, box, main diagonal and anti-diagonal. For every empty cell, it computes the candidate values that do not violate any applicable constraint. It chooses the empty cell with the fewest candidates (minimum remaining values heuristic), tries candidates one at a time, and recursively continues. If a choice leads to a contradiction, the assignment is undone and another candidate is tried.

5. Constraint Propagation and Pruning

Constraint propagation occurs whenever an assigned value is immediately inserted into the relevant row, column, box and diagonal sets. Candidate generation then excludes all values in those sets. The minimum remaining values heuristic prioritizes the most constrained empty cell. A cell with zero candidates causes immediate backtracking, avoiding exploration of a branch that cannot lead to a solution.

6. Correctness

The algorithm is correct because it only assigns values that satisfy every constraint currently applicable to the cell. If a complete grid is produced, every row, column, box and both diagonals contain distinct values from 1 to 9, so the grid is a valid diagonal Sudoku. If a branch cannot be completed, backtracking removes the choice and explores another legal assignment. Therefore, if a solution exists, exhaustive backtracking will find one.

7. Complexity Analysis

For a 9×9 Sudoku, a simple worst-case backtracking search can be described as exponential in the number of empty cells, with an upper-bound intuition of roughly O(9^m), where m is the number of blanks. Candidate checking with sets is approximately O(1) per constraint lookup, so each node can be evaluated efficiently. The diagonal constraints reduce candidate sets and can substantially prune the search tree on particular instances, but they do not change the worst-case exponential nature of general Sudoku solving.

8. Real-World and Practical Relevance

The problem illustrates constraint satisfaction techniques used in scheduling, resource allocation, timetabling, configuration, planning and puzzle solving. The same principles—representing constraints, propagating restrictions, selecting the most constrained variable and pruning infeasible branches—are useful in many discrete optimization systems.

9. Solution Strategy Insights

For small structured puzzles, backtracking with constraint propagation is simple and effective. For harder instances, additional techniques such as forward checking, naked and hidden singles, arc consistency, bit-mask representations, exact-cover formulations, or advanced variable-ordering heuristics can improve performance. The diagonal constraints should be treated as first-class constraints rather than checked only after a full grid has been constructed.

10. Conclusion

Diagonal Sudoku demonstrates how adding constraints changes a classical constraint-satisfaction problem. The two diagonal uniqueness rules interact with rows, columns and boxes and provide additional pruning opportunities. A backtracking solver using candidate propagation and the minimum-remaining-values heuristic provides a correct and practical solution method. Although the search can still be exponential in the worst case, efficient constraint checking and pruning greatly improve practical performance.

11. Python Implementation

```
def solve_diagonal_sudoku(board):
    n = 9
    digits = set(range(1, 10))

    rows = [set() for _ in range(n)]
    cols = [set() for _ in range(n)]
    boxes = [set() for _ in range(n)]
    diag1 = set()
    diag2 = set()

    for r in range(n):
        for c in range(n):
            value = board[r][c]
            if value == 0:
                continue

            b = (r // 3) * 3 + (c // 3)

            if value in rows[r] or value in cols[c] or value in boxes[b]:
                return False

            if r == c and value in diag1:
                return False

            if r + c == n - 1 and value in diag2:
                return False

            rows[r].add(value)
            cols[c].add(value)
            boxes[b].add(value)

            if r == c:
                diag1.add(value)

            if r + c == n - 1:
                diag2.add(value)

    def candidates(r, c):
        b = (r // 3) * 3 + (c // 3)
        used = rows[r] | cols[c] | boxes[b]

        if r == c:
            used |= diag1

        if r + c == n - 1:
            used |= diag2

        return digits - used

    def backtrack():
        best_cell = None
        best_candidates = None
        for r in range(n):
```

```
                for c in range(n):
                    if board[r][c] != 0:
                        continue

                    possible = candidates(r, c)

                    if not possible:
                        return False

                    if best_candidates is None or len(possible) < len(best_candidates):
                        best_cell = (r, c)
                        best_candidates = possible

                        if len(best_candidates) == 1:
                            break
                if best_candidates is not None and len(best_candidates) == 1:
                    break

            if best_cell is None:
                return True

            r, c = best_cell
            b = (r // 3) * 3 + (c // 3)

            for value in sorted(best_candidates):
                board[r][c] = value
                rows[r].add(value)
                cols[c].add(value)
                boxes[b].add(value)

                if r == c:
                    diag1.add(value)

                if r + c == n - 1:
                    diag2.add(value)

                if backtrack():
                    return True

                board[r][c] = 0
                rows[r].remove(value)
                cols[c].remove(value)
                boxes[b].remove(value)

                if r == c:
                    diag1.remove(value)

                if r + c == n - 1:
                    diag2.remove(value)

            return False

        return backtrack()


    if __name__ == "__main__":
        puzzle = [
            [0, 0, 0, 0, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 0, 0, 0],
            [0, 0, 0, 0, 0, 0, 0, 0, 0]
        ]

        if solve_diagonal_sudoku(puzzle):
            for row in puzzle:
                print(*row)
        else:
            print("No solution exists.")
```

12. Algorithm Summary

Step	Operation
1	Validate the initial puzzle against rows, columns, boxes and both diagonals.
2	For each empty cell, compute candidates using all applicable constraints.
3	Choose the cell with the fewest candidates.

4	Assign a candidate and update constraint sets.
5	Recursively continue; immediately reject zero-candidate states.
6	Undo the assignment when a branch fails and try the next candidate.
7	Stop when all cells are filled.